

Falaq Food API Testing

1. Project Overview

This project contains API tests for the Falaq Food ecommerce website using Postman and Newman.

The project covers:

- Product search/autocomplete
- Product selection
- Product retrieval
- Order creation
- Invalid product handling
- Invalid order quantity handling
- Invalid search limit handling
- Invalid product name/search handling
- Response and status-code validation
- Newman execution and HTML reporting

Base URL:

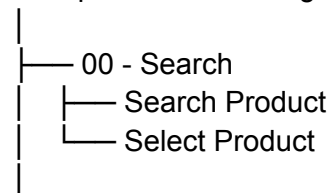
<https://shop.falaqdigital.com>

2. Tools Used

- **Postman** – API requests and test scripts
 - **Newman** – Command-line test execution
 - **Newman HTML Extra Reporter** – HTML test report generation
-

3. Project Structure

Falaq Food API Testing



01 - Products
Get Product
02 - Orders
Create Order
03 - Negative Tests
Get Product - Invalid ID
Create Order - Invalid Quantity
Search Product - Invalid Name
Search Product - Invalid Limit

4. Environment Setup

The project uses the Postman environment:

Falaq Food Env

Main Variable

Variable	Value
base_url	https://shop.falaqdigital.com

Requests use:

{{base_url}}

Make sure **Falaq Food Env** is selected before running the collection in Postman.

For Newman execution, the base URL can also be provided directly:

```
--env-var "base_url=https://shop.falaqdigital.com"
```

5. Test Cases

TC-01 — Search Product

This test verifies the product search/autocomplete API used when searching for a product.

Method: GET

Endpoint:

{{base_url}}/api/v1/storefront/catalog/products/quick-search

Parameters

Parameter	Example
s	methi
limit	8

Example Request

{{base_url}}/api/v1/storefront/catalog/products/quick-search?s=methi&limit=8

Expected

200 OK

Validations

- Status code is 200
- Response is valid JSON
- Response contains an `items` array
- At least one matching product is returned
- Each returned product has an ID
- Each returned product has a name
- Each returned product has a slug
- Each returned product has an image URL
- Each returned product contains at least one variant
- Product variants contain valid prices

Purpose

This test verifies that the API successfully returns matching products when a valid search term is provided.

TC-02 — Select Product

This test verifies that a product can be retrieved using its slug after selecting it from the search results.

Method: GET

Endpoint:

{{base_url}}/api/v1/storefront/catalog/products/slug/{slug}

Example Request

{{base_url}}/api/v1/storefront/catalog/products/slug/methi-mix

Expected

200 OK

Validations

- Status code is 200
 - Response is valid JSON
 - Product information is returned
 - Product slug is correct
 - Product contains required product details
 - Product contains at least one variant
-

TC-03 — Get Product

This test verifies that a product can be retrieved using its product ID.

Method: GET

Endpoint:

{{base_url}}/api/v1/storefront/catalog/products/{productId}

Expected

200 OK

Validations

- Status code is 200

- Response is valid JSON
 - Product ID is correct
 - Product status is Active
 - Product has at least one variant
-

TC-04 — Create Order

This test verifies that a valid order can be successfully created.

Method: POST

Endpoint:

{{base_url}}/api/v1/storefront/orders

The request contains customer details, product/variant information, quantity, and shipping address.

Expected

201 Created

Validations

- Status code is 201
- Response is valid JSON
- Order ID is present
- Order number is present
- Order number follows the expected format
- Order total is greater than zero
- Order creation time is present
- Account status is present

The following response values are also saved for potential future use:

order_id
order_number
order_total

Note: This endpoint creates an actual order, so it should only be executed against the production site when authorized.

6. Negative Tests

TC-05 — Get Product - Invalid ID

This test verifies how the API handles a request for a product that does not exist.

Method: GET

Endpoint:

{{base_url}}/api/v1/storefront/catalog/products/00000000-0000-0000-0000-000000000000

Expected

404 Not Found

Example Response

```
{
  "title": "Resource not found.",
  "status": 404
}
```

Validations

- Status code is 404
 - Response is valid JSON
 - Error title is Resource not found.
 - Response status is 404
-

TC-06 — Create Order - Invalid Quantity

This test checks how the API handles an invalid order quantity.

Method: POST

Endpoint:

{{base_url}}/api/v1/storefront/orders

The request uses:

```
{  
  "quantity": 0  
}
```

instead of a valid positive quantity.

Expected

The API should reject the invalid quantity rather than create an order.

The exact status code and validation response should match the API's validation behavior.

Purpose

This test verifies that the API does not accept an order containing an invalid quantity.

Note: This test should validate the API's actual error response for `quantity: 0` rather than expecting `201 Created`.

TC-07 — Search Product - Invalid Name

This test verifies how the search API handles an invalid or non-existing product name/search term.

Method: GET

Endpoint:

{{base_url}}/api/v1/storefront/catalog/products/quick-search

Parameters

Parameter	Example
s	Invalid/non-existing product name
limit	8

Expected

200 OK

The API should successfully process the search request but return no matching products.

Validations

- Status code is 200
- Response is valid JSON
- Response contains an `items` array
- `items` array is empty when there is no matching product

Purpose

This test verifies that an invalid or non-existing product name does not return unrelated products.

TC-08 — Search Product - Invalid Limit

This test verifies how the search API handles an invalid value for the `limit` parameter.

Method: GET

Endpoint:

`{{base_url}}/api/v1/storefront/catalog/products/quick-search`

Example Request

`{{base_url}}/api/v1/storefront/catalog/products/quick-search?s=methi&limit=-1`

Expected

The API should handle the invalid limit according to its validation/business rules.

The test should verify the actual response behavior, including the returned status code and response structure.

Validations

- API does not return an unexpected server error
- Response behavior is consistent with the API's handling of invalid limits
- Response is valid JSON when a response is returned
- Returned product list follows the API's defined limit behavior

Purpose

This test verifies that invalid values supplied to the `limit` parameter are handled safely and do not cause unexpected API behavior.

7. Test Execution

The collection can be executed through Postman or Newman.

Postman

Select the **Falaq Food Env** environment and run the collection.

Newman

```
newman run "Falaq Food API Testing.postman_collection.json" --env-var  
"base_url=https://shop.falaqdigital.com"
```

Newman HTML Report

```
newman run "Falaq Food API Testing.postman_collection.json" --env-var  
"base_url=https://shop.falaqdigital.com" -r htmlextra --reporter-htmlextra-export  
"Falaq-Food-API-Report.html"
```

The HTML report provides details about:

- Total requests executed
- Passed and failed tests
- Assertion results
- Response details
- Execution time
- Failed test information